

計算機概論（二）

Project1

Pokémon RPG (寶可夢)

Report



姓名：彭靖凱、丁宗弘

學號：E24134778、E24136128

授課教授：陳盈如 教授

實作說明 (Implementation Description)

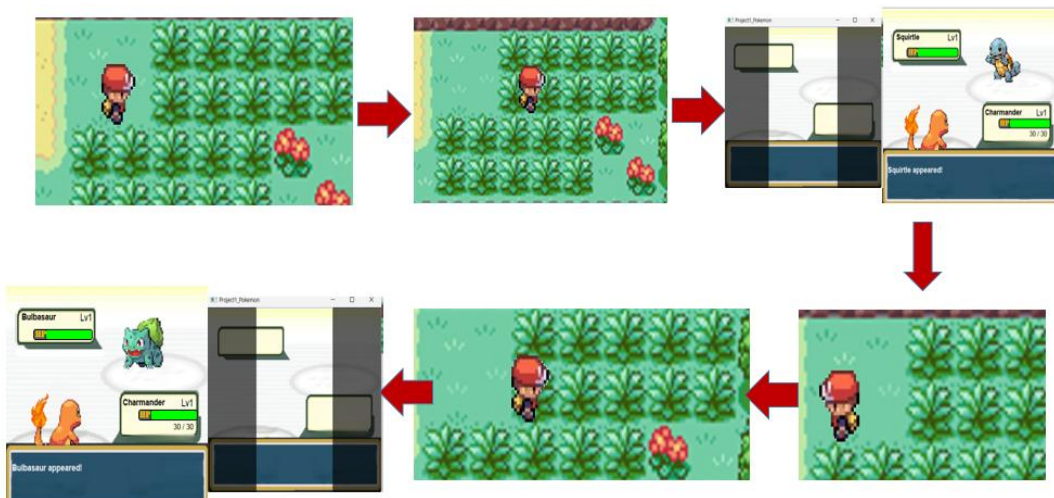
本專案為一款以寶可夢為題材的 2D 回合制角色扮演遊戲 (RPG)，由玩家扮演訓練家，探索草地、城鎮與實驗室，與 NPC 互動並捕捉野生寶可夢。本遊戲採用 Qt 框架開發，以 QGraphicsScene 為核心，實作場景切換、互動事件與戰鬥系統。

1. 遊戲世界與探索邏輯

遊戲開場會顯示啟動畫面，進入主城鎮 (Pallet Town) 後，玩家可自由移動，並與地圖上的公告欄、NPC、寶箱等物件互動。實驗室內可選擇初始寶可夢，三選一後正式展開冒險。



草地區域設有高草區 (Tall Grass)，玩家進入後將依機率觸發隨機戰鬥。每個草區只能觸發一次戰鬥，離開並重新進入才會再次觸發。



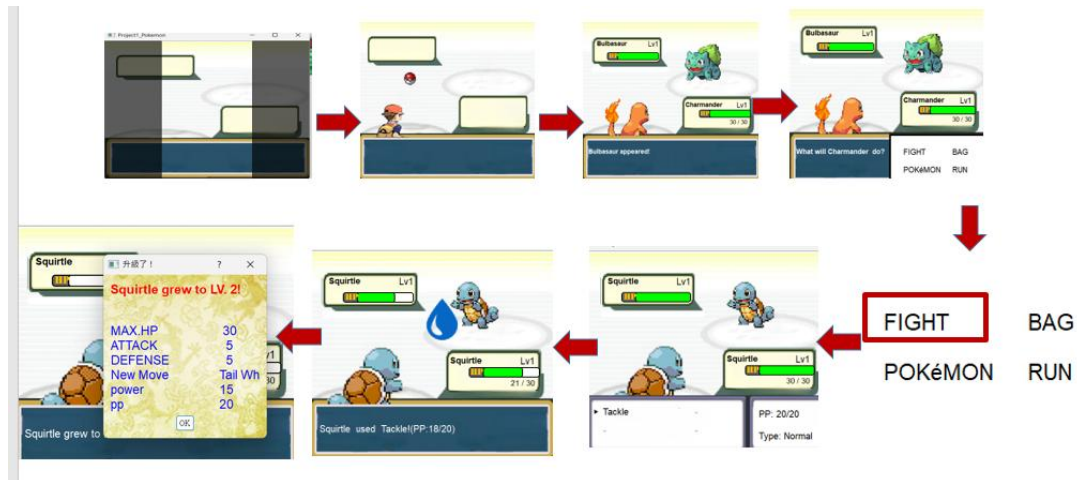
2. 戰鬥流程設計

戰鬥為典型的回合制系統。當進入戰鬥場景後，畫面會顯示敵方野生寶可夢與

我方出戰寶可夢，並提供四個選項：

- (1) FIGHT：選擇技能進行攻擊，技能具有屬性、PP、威力與分類
- (2) BAG：使用道具（如 Potion、Ether、Poké Ball）
- (3) POKéMON：切換上場寶可夢
- (4) RUN：嘗試逃跑

每個技能使用一次會消耗 1 PP。當敵方寶可夢被擊敗後，玩家寶可夢會獲得經驗值並升級，特定等級可觸發學習新技能或進化。



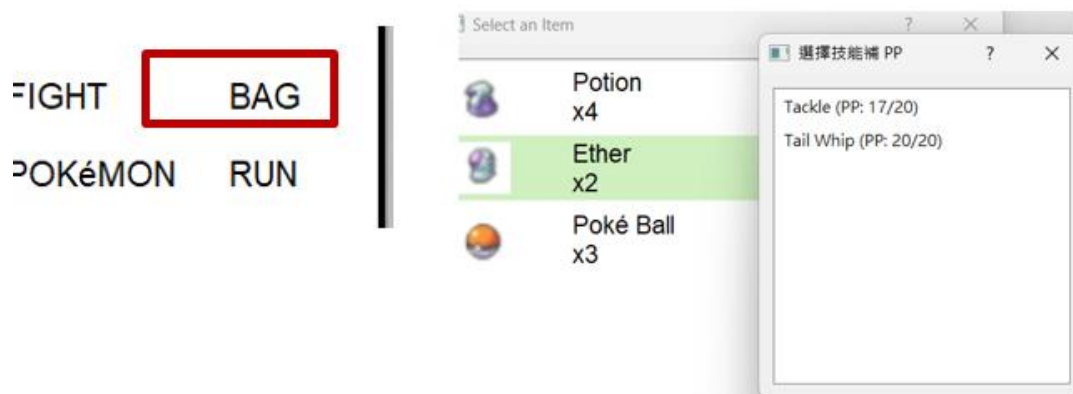
3. 捕捉機制

當使用 Poké Ball 時，會播放投擲動畫並進行捕捉判定。成功則新增寶可夢至隊伍，並更新背包中 Poké Ball 數量。捕捉機率根據對手 HP 與基礎捕捉率決定。



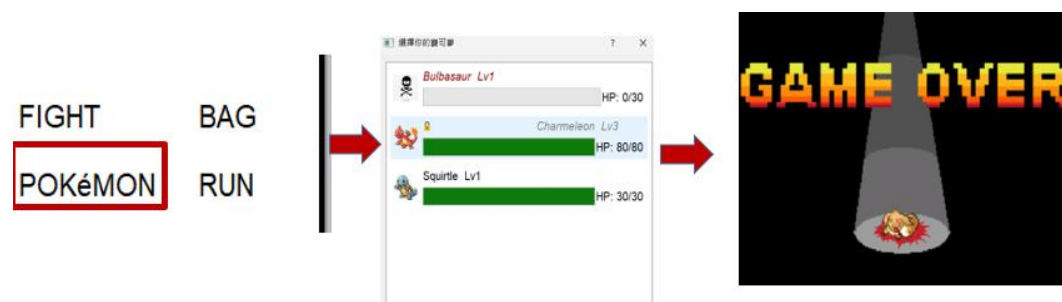
4. 道具與技能管理

- (1) Potion：回復寶可夢 HP
- (2) Ether：補充技能 PP（透過滑鼠點選目標技能）
- (3) 精靈球：嘗試捕捉對方寶可夢
- (4) 技能顯示：包含屬性圖示、剩餘 PP 與技能分類



5. 寶可夢切換與死亡邏輯

玩家可於戰鬥中打開隊伍選單，檢視每隻寶可夢的等級與 HP。若寶可夢死亡（HP 為 0），會顯示骷髏圖示並禁止選用。若隊伍全數陣亡，則顯示 Game Over 畫面。



寶可夢技能表與屬性相剋（含寶可夢屬性）

技能、寶可夢屬性與分類

寶可夢	屬性	等級	技能名稱	技能屬性	分類	威力	PP
Charmander	Fire	1	Scratch	Normal	Physical	10	20
Charmander	Fire	2	Growl	Normal	Status	15	20
Charmeleon	Fire	3	Scary Face	Normal	Status	20	15
Charmeleon	Fire	4	Flare Blitz	Fire	Physical	25	5
Bulbasaur	Grass	1	Tackle	Normal	Physical	10	20
Bulbasaur	Grass	2	Growl	Normal	Status	15	20
Ivysaur	Grass	3	Growth	Normal	Status	20	15
Ivysaur	Grass	4	Razor Leaf	Grass	Physical	25	5
Squirtle	Water	1	Tackle	Normal	Physical	10	20

Squirtle	Water	2	Tail Whip	Normal	Status	15	20
Wartortle	Water	3	Protect	Normal	Status	20	15
Wartortle	Water	4	Wave Crash	Water	Physical	25	5

屬性相剋對照表（加分項目）

攻擊技能屬性	剋制對象(寶可夢)
Fire	Grass
Water	Fire
Grass	Water
Fire	Water
Water	Grass
Grass	Fire

系統架構設計

本專案這是一款基於 Pokémon 世界觀的簡易角色扮演遊戲 (RPG)，玩家可以探索世界、捕捉寶可夢、進行回合制戰鬥並養成隊伍。

使用 **Qt Framework** 開發，核心採用 QGraphicsView + QGraphicsScene 架構實作遊戲畫面與邏輯控制。整體系統以模組化方式進行，將地圖、角色、戰鬥、資源、聲音等分別交由獨立類別負責，易於維護與擴充。

1. 程式架構總覽

主程式 (main.cpp) 負責啟動視窗與初始化。

View (view.cpp/h) 為主要視圖，控制顯示內容。

GameScene (gamescene.cpp/h) 管理遊戲場景的更新與渲染。

(1) void GameScene::update() — 探索場景更新

這是主場景的更新函式，由 QTimer 週期性觸發。主要負責處理以下邏輯：

- 更新背景與角色移動狀態，並繪製目前地圖畫面
- 判斷是否進入戰鬥場景 (SceneIndex==3)，若是則啟動畫面過場動畫與設定戰鬥參數
- 碰撞判斷與事件觸發（例如觸發高草區遇敵、進入門、接近 NPC 等）
- 若處於探索模式中，則持續刷新玩家動畫、背景與地圖物件

這個函式就是「整個遊戲非戰鬥狀態下的主循環邏輯」。

(2) void GameScene::updateBattle() — 戰鬥畫面更新

此函式由 m_battleTimer 定期呼叫，專門在戰鬥場景中運作。它負責：

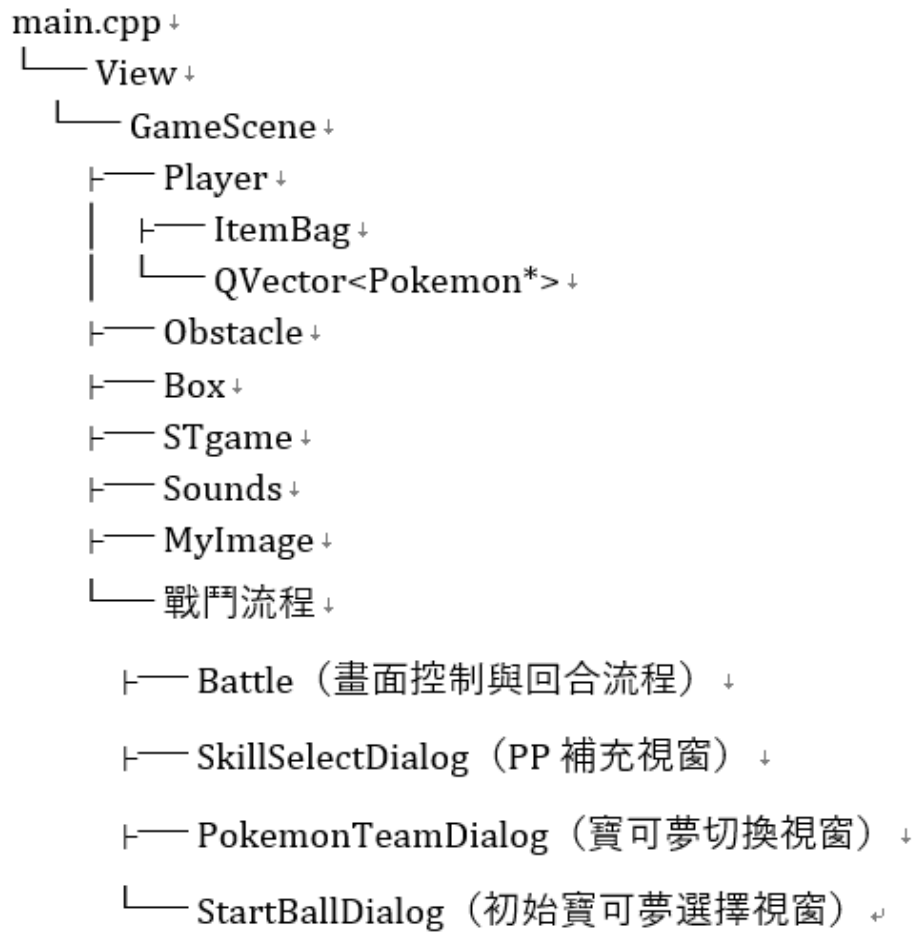
- **畫面清空與重繪背景與寶可夢圖像**
- 若正在捕捉寶可夢（tryCatchPokemon_flag 為真），播放 Poké Ball 投擲動畫與命中邏輯
- **回合制戰鬥控制流程**：交替執行玩家與敵方攻擊
 - 玩家回合：選擇技能 → 使用技能 → 計算傷害 → 顯示訊息
 - 敵人回合：自動使用技能攻擊 → 檢查是否造成我方寶可夢陣亡
- 若寶可夢死亡 → 觸發切換、升級或結束戰鬥邏輯
- 若戰鬥結束 → 清除戰鬥場景並返回探索地圖

此函式是「完整戰鬥邏輯的主控中心」，涵蓋了動畫、事件邏輯與 UI 顯示等所有處理。

Pokémon RPG — 模組總表

模組名稱	功能簡介
main.cpp	程式入口點，包含啟動畫面動畫邏輯與 Qt 執行環境初始化
View	使用 QGraphicsView 顯示主畫面，載入並顯示 GameScene
GameScene	管理所有場景切換（城鎮、草原、實驗室、戰鬥），並處理玩家移動與互動
Player	管理玩家位置、方向、寶可夢隊伍、道具背包與碰撞邏輯
Pokemon	實作寶可夢資料結構（名稱、等級、HP、攻擊、防禦、技能、屬性）
Battle	處理回合制戰鬥畫面與流程（技能選擇、道具使用、傷害計算等）
ItemBag	管理所有可使用道具的種類與數量
Obstacle	控制場景中所有障礙物與碰撞框架邏輯
Box	實作場景中可互動的寶箱系統（隨機掉落道具）
STgame	管理遊戲整體狀態，如場景 index、寶可夢生成、寶箱隨機生成邏輯等
Sounds	播放不同場景或事件對應的背景音樂與音效
MyImage	載入與管理所有圖片資源，包括場景圖、角色圖、技能圖示等
StartBallDialog	初始寶可夢選擇視窗，顯示三隻初始寶可夢圖像與名稱，點選後進入遊戲
SkillSelectDialog	補 PP 選擇視窗：顯示所有技能與目前/最大 PP，選擇要補充的技能
PokemonTeamDialog	寶可夢切換視窗，顯示隊伍寶可夢狀態與死亡圖示，支援選擇與切換

模組層級結構（main.cpp → View → GameScene）



模組	說明
main.cpp	程式執行入口：建立 View 物件並顯示遊戲主畫面
View	繼承自 QGraphicsView，負責顯示整體視窗畫面與載入 GameScene
GameScene	繼承自 QGraphicsScene，為主要邏輯場景：處理鍵盤輸入、物件顯示、事件觸發等

這個流程的意義在於：**main.cpp** 不直接管邏輯，只負責顯示 **View**，而 **View** 又把遊戲邏輯交給 **GameScene** 管理，這樣可以讓顯示與邏輯分離，

STgame 成員屬性階層

```
STgame ↓
├── Player ↓
├── m_Battle ↓
│   ├── m_playerPokemonSprite → 顯示目前寶可夢圖像 (QPixmap) ↓
│   └── m_MyImage ↓
│       └── m_ballBgPixmap → 背景精靈球圖片 (QPixmap) ↓
├── m_playerTeam → 玩家隊伍資料 (QVector<Pokemon*>) ↓
└── m_StevenPeng ↓
    └── Bag_pokemonTeam → 特定角色的寶可夢隊伍 (QVector<Pokemon*>) ↓
```

遇到的挑戰與解決方式

在本次專案中，我遇到不少挑戰。首先，我過去沒有使用 C++ 開發過遊戲，也完全沒接觸過 Qt 架構與 Qt GUI 系統，更不熟悉電腦視覺中「鏡頭與畫面、視窗管理」的概念。因此，一開始面對這個作業時感到相當惶恐。

開發過程主要是透過上網查資料、閱讀教學、並不斷與同學、家人討論來學習。Qt 的學習曲線不低，但透過實際操作，我逐漸理解了視窗物件、場景切換與事件傳遞的邏輯，並培養了程式邏輯與排錯能力。

其中一個具體的挑戰是 NPC 的移動與碰撞檢測。一般障礙物只需檢查是否與玩家位置相交即可，但 NPC 是會移動的，彼此之間可能也會產生碰撞。我必須進一步處理雙方互動與避免重疊的情況，這比預期的難度更高。

另外，寶可夢的資料在多個對話框與場景間的傳遞也是一個困難點。資料容易遺失，尤其在不同的視窗或物件之間切換時。為此我不斷嘗試使用「傳值」、「指標」、「參考」等不同方式傳遞資料，最終找出一個穩定的解法，能正確同步寶可夢狀態。

一開始我將探索場景與戰鬥場景設計在同一個畫面中，但隨著功能複雜度提高，發現這樣很難管理邏輯與 UI。後來我選擇將兩者分開管理成不同的場景（Scene），這樣在事件監聽、畫面更新與資料管理上都更加清晰與穩定。另外遊戲中其他問題

1. 場景碰撞與障礙物穿越問題

問題： 玩家角色初期能夠穿越書櫃、NPC、懸崖等障礙物，造成不合理操作。

解法： 我們為所有障礙物建立 QRectF 邊界，並在每次移動時執行 CheckObstacle() 檢查是否重疊，若有碰撞即取消移動，確保無法穿越。

2 高草區連續遇敵問題

問題： 玩家在高草區內移動會每幾步就重複觸發戰鬥，導致體驗不佳。

解法： 為每個草區建立 visited 狀態標記，進入後只允許觸發一次遇敵，離開後才重置，達到區域觸發一次的效果。

3. 傷害計算缺乏屬性影響

問題： 初期所有技能傷害一致，缺乏屬性相剋感，遊戲策略性不足。

解法： 加入火、草、水三種屬性傷害倍率邏輯，並嵌入於

Pokemon::calculatePowerDamage() 中，自動根據技能與對手屬性套用傷害倍率。

總結來說，這次作業不僅讓我學會了 Qt 與 C++ 的應用，更重要的是訓練了我分階段拆解問題與逐步解決的能力，也讓我更有信心面對未來的期末專案。